

TRIE-AWARE TOOL MEMORY · FYP RESEARCH REPORT

Initial Research Findings

Task F deliverable: dataset inventory, schema-length and frequency figures, prefix-cache sanity results, ordering comparisons, and the recommendation for the next research stage.

● Initial experiments measured · Local audit complete

45,815

CANONICAL TOOLS

9,201

BENCHMARK TASKS

10.6×

TTFT GAIN AT 200
TOOLS

31.80%

BEST TOOLRET
ESTIMATE

18.88%

BEST BFCL ESTIMATE

CONTENTS

Result in one paragraph

Evidence obtained locally

Prefix-cache sanity results

Prefill cost and the measurement floor

Analytical estimate versus measured cache hits

Does the winning ordering preserve function-call quality?

Does request order matter on the live GPU cache?

Retrieved-menu GPU arm

Exact rendered-token and memory audit

ContextPilot confirmation and dual-model replication

Recommendation

Initial-brief execution status

Executive answer

Public tool workloads contain a measurable analytical prefix-locality signal, but it is workload- and evidence-dependent. Deterministic ordering raises estimated block reuse over empirical order for both ToolRet and BFCL. Measured on vLLM, prefix caching yields no TTFT benefit at 303-token prompts but up to a 10.6x reduction on padded shared-catalog menus. On deterministic BM25-retrieved menus, ToolTrie adds only 0.76-1.65 percentage points of reuse without a resolved TTFT gain. A fresh 4B-primary/0.6B-replication matrix finds that both corrected ContextPilot adaptations lead ToolTrie on every retrieved menu size, while all three lose most reuse as menu overlap falls. The recommendation is to keep ordinary text prefill as fallback and treat safe retention as a separate extension.

Result in one paragraph

The public workloads contain a measurable analytical prefix-locality signal, but it is workload- and evidence-dependent. Across 45,815 canonical tools, median schema length is 70 Qwen tokens and P95 is 220. ToolRet's median relevance set and BFCL's median exposed menu both contain one tool, which limits what reordering can achieve on many tasks. On empirical dataset order, schema-cost-weighted ordering raises the ToolRet tool-prefix block-reuse estimate from 26.91% to 31.80% (4.89 percentage points). Frequency ordering raises the BFCL menu estimate from 10.98% to 18.88% (7.90 points). These are analytical estimates, not vLLM hit-rate or latency results.

Evidence obtained locally

- ToolRet: 7,961 tasks, 7,652 tools appearing in gold relevance labels, with every label resolved.
- BFCL subset: 1,240 tasks and 1,362 distinct canonical functions appearing in exposed menus.
- Schema diagnostics flag 19,921/45,815 tools. Most flags are empty parameter objects, which can be legitimate; malformed and missing-field counts are reported separately.
- Controlled support-skewed replay produces much more reuse than uniform or empirical access, but under unbounded retention that gap comes from resampling with replacement — repeated requests — not from locality. Only the finite-cache tables in `access-patterns.md` speak to request ordering.
- Classic FP-tree global order equals the frequency order in this baseline. Conditional pattern mining is still untested.

Prefix-cache sanity results

Measured on Qwen/Qwen3-0.6B, block size 16, 11807 GPU blocks, 5 trials per scenario with the prefix cache reset before each trial. Intervals are 95% Student-t half-widths.

CHECK	CACHED / PROMPT TOKENS	REUSE	TTFT ON (MS)	TTFT OFF (MS)
Cold prompt	0 / 303	0.00%	52.4 ± 28.8	51.5 ± 27.3
Identical prompt reuse	288 / 303	95.05%	43.0 ± 1.2	40.5 ± 2.3
Changed second tool	128 / 309	41.42%	41.4 ± 1.1	42.3 ± 2.2
Reordered first two tools	48 / 303	15.84%	37.7 ± 5.7	43.0 ± 4.2
Original restored	288 / 303	95.05%	41.1 ± 2.1	39.4 ± 2.8

The control served `enable_prefix_caching=False` and reported 0 cached tokens in every scenario, so the cache-on/cache-off output comparison is meaningful.

Prefill cost and the measurement floor

Menus are one gold tool padded with distractors from a fixed global catalog. Cold is measured after a prefix-cache reset; warm is an identical repeat. 5 trials per point.

MENU TOOLS	PROMPT TOKENS	COLD TTFT ON (MS)	WARM TTFT ON (MS)	WARM TTFT OFF (MS)	SPEEDUP
1	250	25.6	23.4	23.1	1.0x
4	433	25.2	16.9	23.7	1.4x
16	1,771	62.1	35.3	56.5	1.6x
64	6,742	248.8	54.1	245.6	4.5x
128	13,422	647.8	85.2	640.7	7.5x
200	20,627	1282.7	120.7	1275.2	10.6x

Prefix caching first produces a material gain at **4 tools** and grows steeply from there.

The cache-disabled control shows cold and warm within noise of each other at every size, so the gap above is prefix caching and not a warmup artifact. Note that within-run cold-vs-warm separation alone is not sufficient evidence: the control also 'separates' at one tool, where no cache exists. The cache-on/cache-off comparison is the trustworthy one.

This supersedes the earlier reading that prefix reuse buys nothing. It buys nothing at 303 tokens, which is simply below the floor.

Analytical estimate versus measured cache hits

WORKLOAD	PREDICTED CACHEABLE	MEASURED CACHED	MEASURED/PREDICTED	PREDICTED REUSE	MEASURED REUSE
menu64-alphabetical	428,016	526,432	1.23x	38.42%	38.15%
menu64-frequency	40,448	60,864	1.50x	3.63%	4.41%

The tool-unit model under-predicts rather than over-predicts. It ignores the chat template and system preamble, which are identical across requests and are themselves cached, and that outweighs the partial blocks it loses at tool boundaries.

The ordering comparison inverts the gold-only recommendation. On padded menus, ordering by benchmark support puts the *task-specific* tools first and pushes the shared catalog behind them, destroying the common prefix. A stable global order that ignores task-specificity keeps the shared catalog in front. Frequency ordering is the right choice only when the whole menu is task-specific, which is not what a connected tool catalog looks like.

Does the winning ordering preserve function-call quality?

100 BFCL tasks (20 per category, 64-tool menus), scored against BFCL `possible_answer` ground truth. Alphabetical is the ordering measured strongest for cache reuse; frequency is the comparison Task D's analytical model originally recommended.

Qwen/Qwen3-0.6B :

ORDERING	FUNCTION-NAME ACCURACY	FULL ACCURACY	NO-TOOL ACCURACY
alphabetical	67.50%	47.50%	95.00%
frequency	77.50%	51.25%	85.00%

Qwen/Qwen3-8B, identical workload:

ORDERING	FUNCTION-NAME ACCURACY	FULL ACCURACY	NO-TOOL ACCURACY
alphabetical	91.25%	81.25%	95.00%
frequency	91.25%	81.25%	90.00%

The name/full-accuracy gap that looked real at 0.6B is not present at 8B — both orderings score identically there. Checking a counterintuitive result against a second model found that the apparent quality tradeoff is model- or sample-sensitive; only a smaller no-tool-accuracy gap favouring alphabetical survives at both checkpoints. One run per condition, no repeats; read as directional rather than as a general model-size law.

Does request order matter on the live GPU cache?

Every other GPU experiment above resets the prefix cache before each trial, which is repeatable but erases the cross-request dependency locality is actually about. This instead runs one continuous session per replay condition — a single reset, then every request in sequence — comparing `empirical` against `session_bursty`, the one replay pair that is a strict permutation of the same task multiset.

REPLAY	CANONICAL TOOL TOKENS	MEASURED CACHED TOKENS	CACHED / CANONICAL DIAGNOSTIC	PREDICTED REUSE (BOUNDED TRIE)	PREDICTED EVICTIONS
empirical	668,440	287,200	42.97%	35.00%	2,927
session_bursty	668,440	287,616	43.03%	35.03%	2,918

The original script mislabeled canonical tool-token volume as full prompt tokens and used it as the denominator. Therefore 42.97%/43.03% are **not** standard rendered-prompt cache-hit ratios. The measured cached-token totals are valid and differ by only 416 tokens (0.14%), which still supports the narrow finding that these two request orders behaved almost identically.

The volume exceeded the nominal 190,896-token cache capacity and the offline model predicted eviction, but this run did not sample live KV occupancy or an engine eviction counter, so it must not be cited as direct memory-pressure evidence. The corrected `locality_replay.py` now uses rendered prompt tokens, validates counters, samples occupancy, and can require a pressure threshold.

Retrieved-menu GPU arm

A deterministic BM25 baseline selected each ToolRet menu without reading gold IDs; gold labels were used only for the separate retrieval evaluation. All ordering conditions used the same selected tool membership and request sequence. Each cache result below is three clean 200-request trials on Qwen3-0.6B/vLLM.

MENU K	MACRO RECALL	HIT RATE	MRR
4	41.71%	51.50%	0.3858
16	55.04%	65.50%	0.4022
64	64.21%	75.50%	0.4061
128	67.54%	80.50%	0.4066

K	ORIGINAL FALLBACK	ALPHABETICAL	RANDOM SEED 42	FREQUENCY	SCHEMA-COST WEIGHTED	FP-TREE GLOBAL	TOOLTRIE-V0
4	15.87%	15.28%	14.17%	14.62%	14.84%	14.62%	17.48%
16	6.12%	6.27%	5.39%	5.59%	6.24%	5.59%	7.77%
64	0.91%	1.24%	0.96%	0.96%	1.09%	0.96%	1.90%
128	0.37%	0.58%	0.59%	0.54%	0.57%	0.54%	1.13%

K	PROMPT TOKENS/QUERY	FALLBACK REUSE	BEST STATIC REUSE	TOOLTRIE REUSE	TOOLTRIE - FALLBACK	FALLBACK TTFT/QUERY (MS)	TOOLTRIE TTFT/QUERY (MS)
4	640	15.87%	15.87%	17.48%	+1.61 pp	32.3	31.2
16	2,139	6.12%	6.27%	7.77%	+1.65 pp	70.7	70.3
64	8,361	0.91%	1.24%	1.90%	+0.99 pp	349.7	350.0
128	16,267	0.37%	0.59%	1.13%	+0.76 pp	937.0	933.5

ToolTrie-v0 has the highest reuse at every retrieved menu size, but the absolute advantage over ordinary retrieval-rank text prefill is only 0.76-1.65 percentage points. Reuse falls as menus grow because independently retrieved sets share little exact rendered prefix. The three-trial TTFT intervals overlap and no paired-difference interval was predeclared, so these data do not establish a TTFT improvement.

At k=64, both predeclared direct partial-reuse strata contain 199 requests per trial (plus one cold request). Alphabetical measured 1.37% reuse and 351.7 ms TTFT; ToolTrie measured 2.12% and 350.5 ms. This is direct partial-reuse evidence, but the latency difference is too small to support a speed claim.

Exact rendered-token and memory audit

All 7/7 k64 rendered-prefix audits are clean (`all_clean=true`). The server rendered and tokenized the same chat-plus-tools payload used for completion; token counts and cached-plus-computed identities match for every request. Exact token IDs and block boundaries remain in the checksummed raw archive rather than Git.

The first pressure attempt produced 24 clean, reset regime-runs but accepted 0/24: sampled occupancy was only 3.64-3.69% in a 190,896-token cache. The sequential client held one approximately 7k-token request resident; cumulative prompt volume is not residency. These runs are preserved and quarantined from pressure claims.

The separately predeclared controlled-cache rerun is fully accepted: 24/24 regime-runs and 384/384 checks passed at exactly 7,680 tokens. Peak occupancy was 91.02-91.86% against the unchanged 90% threshold, with 57,696-85,340 sampled evictions, zero preemptions, and one running/zero waiting requests throughout.

CONTROLLED- PRESSURE ORDERING	EMPIRICAL REUSE	UNIFORM REUSE	SKEWED REUSE	SESSION- BURSTY REUSE	EVICTIONS
Original	1.18%	0.69%	1.24%	0.69%	84,810- 85,340
Alphabetical	29.21%	29.35%	28.06%	27.76%	60,600- 61,948
Random seed 42	32.16%	31.27%	32.67%	30.19%	57,696- 59,854
Frequency = schema-cost = FP-tree	9.44%	8.99%	9.51%	9.00%	77,682- 78,176

This is a within-capacity stress comparison, not a latency or production-pressure result. Random seed 42 ranks first in all four regimes, but there is only one run per cell and one fixed random seed, with no uncertainty interval. That is insufficient to recommend random ordering. A deterministic reconstruction finds 4 distinct orders: frequency, schema-cost weighted, and FP-tree global emit exactly the same 200 sequences in every regime. They are one equivalence class here, not three independent policies.

ContextPilot confirmation and dual-model replication

A pinned confirmation run used upstream commit `1fa0a143fdeda344585666648ab2b30cb7fea77f`, the paper/default `alpha=0.001`, one persistent `ContextPilot.reorder` index, stock vLLM, and the same selected tool sets and request sequence. This is an **ordering-only persistent-API adaptation**: workloads were planned before serving and no eviction feedback, relevance annotations, or de-duplication were enabled. It is not the full ContextPilot system.

On padded 64-tool menus it reaches 96.16% BFCL and 95.27% ToolRet reuse versus 87.19% and 83.58% for ToolTrie-v0. On BM25-retrieved ToolRet menus it also leads ToolTrie at every measured size, but absolute reuse falls to 18.72%, 9.93%, 4.78%, and 1.99% at k=4/16/64/128. The 95–96% headline is therefore a shared-menu positive-control result rather than a realistic retrieval result.

The Qwen3-8B quality replay gives the persistent arm +2.03 points on function name, +2.03 on full call, and –5.00 on no-tool accuracy against alphabetical on one fixed request sequence. An earlier Qwen3-4B addendum found 0.00 points on no-tool; the fresh 4B-primary dual-model run finds –0.62 points with an interval spanning zero. The current evidence supports a model-sensitive quality frontier, not a universal safety penalty.

The fresh predeclared matrix accepts all 190 replays across Qwen3-4B primary and Qwen3-0.6B replication. Both the persistent-API and corrected static-refit adaptations beat ToolTrie-v0 reuse in all 12 model/workload cells, but none is a universal quality winner. See `reports/contextpilot-confirmation/findings.md` and `reports/contextpilot-dual-model/findings.md`.

The separate static-refit resume is now integrated: 18/18 systems trials, one Qwen3-8B quality replay, and the independent 72/72 SGLang aggregate-counter audit pass their compact checks. The two ContextPilot APIs have identical 8B aggregate and paired statistics. That historical 8B quality matrix still lacks the `original` fallback, so its gain versus alphabetical is not a measured gain over ordinary selected-tool text prefill.

A causal `frequency_online` ablation reaches 96.27% on BFCL padded-64 by identifying the 63 tools present in every request. This demonstrates that the padded positive control cannot distinguish a trie or clusterer from a simple counter. It beats ToolTrie-v0 in 10/12 cells, not the 12/12 originally claimed, and loses both BM25 k=4 cells (17.01% versus 17.48%). It has no quality replay.

Recommendation

The retrieved-menu arm changes the recommendation. Alphabetical remains a strong simple baseline on the padded shared-catalog workload, but it is not a universal winner once menu membership comes from retrieval. ToolTrie-v0 is the strongest in-brief ordering baseline at all four retrieved menu sizes, but both corrected ContextPilot adaptations exceed it in the fresh external comparison. The absolute gains remain small and do not produce a resolved TTFT improvement. Retrieval coverage and selected-set overlap are now the dominant bottlenecks: increasing the BM25 menu from 4 to 128 raises macro recall from 41.71% to 67.54%, while prompt cost grows about 25x and exact reuse falls.

Under the separately controlled 7,680-token pressure condition, fixed random seed 42 measures the most reuse and the fewest evictions in all four regimes, ahead of alphabetical. This ranking is capacity- and workload-specific, and a single run per cell and random seed is not a policy result; run repeated seed sweeps before treating it as more than a useful sensitivity finding.

The likely publishable refinement is not a generic "reorder context into a trie" claim, because closely related cache-aware context ordering already exists and the corrected ContextPilot persistent-API adaptation beats ToolTrie-v0 on every padded and retrieved reuse cell measured. Both lose most reuse on independently retrieved large menus, so retrieval overlap is the stronger systems bottleneck. The historical static-refit arm and the persistent arm must retain their exact labels; neither is full ContextPilot. The corrected `alpha=0.001` static-refit adaptation is measured in the dual-model run and its separate historical 8B cell is now tracked. The independent SGLang counter audit also accepts 72/72 raw runs. Counter cleanliness does not imply that every policy label is distinct: all five fitted BFCL labels emit one byte-identical sequence.

Do not pursue arbitrary independent KV concatenation yet. Native exact APC already converts the local token-reuse signal into a repeatable TTFT benefit (10.6x at 200 tools). Phase 2 subsequently detected a fixed-sequence no-tool regression among the measured high-reuse orders, so this report must not describe the optimization as quality-preserving without that qualification. Keep ordinary selected-tool text prefill as the explicit default fallback unless a predeclared cost model predicts that reuse repays retrieval, context, decode, and safety costs.

Initial-brief execution status

The explicit Tasks A-F and the stricter gap-closure manifest have been substantively executed. Seven initial experimental questions are answered and question 4 is partial: schema-token cost weighting was measured, but a policy weighted by separately measured per-schema prefill time was not. The evidence includes the ordinary fallback, BM25-retrieved menus, direct partial reuse, rendered- token/block validation, and 24/24 accepted controlled-pressure regime-runs with observed evictions. The original 0/24 runs remain preserved as valid low- occupancy evidence rather than being rewritten.

This completes the planned initial experiment stage. The local analysis and artifact-to-report audit are complete, and the fresh 190-replay dual-model matrix is accepted for its declared scope. Operational closure still needs off-machine archive backup. The historical Qwen3-8B table lacks `original`, but that server command did not pin a model revision; one appended row is controlled only if the exact historical snapshot can be proved. Otherwise use a fresh pinned five-condition matrix or omit that optional 8B comparison. The accepted 4B-primary table already includes the ordinary fallback. Later §9 retained-tool or KV-composition experiments remain extensions and must preserve that fallback, include `frequency_online` as an ablation, and report the measured quality/safety frontier.